

Chương này trình bày chi tiết các giải pháp kỹ thuật, phương pháp tiếp cận và những đóng góp cốt lõi đã đạt được trong quá trình thực hiện đồ án tốt nghiệp. Đây là thành quả của việc tự nghiên cứu, ứng dụng và tinh chỉnh các kiến trúc thành giải pháp thực tiễn cho dự án "Paradox Caliber". Các điểm nhấn đóng góp bao gồm nhiều khía cạnh khác nhau như: thiết kế tùy biến và làm chủ hệ thống cổng không gian, cấu trúc thành phần điều khiển vũ khí, tổ chức hệ thống nhiệm vụ mở rộng linh hoạt, cùng nhiều cơ chế hệ thống khác.

0.1 Tích hợp và tùy biến hệ thống cổng không gian

0.1.1 Dẫn dắt vấn đề

Cơ chế cổng không gian là một tính năng phức tạp để phát triển do đòi hỏi sự kết hợp chặt chẽ giữa toán học không gian (ma trận, véc-tơ chiều), xử lý vật lý (bảo toàn động lượng cơ thể) và kỹ thuật kết xuất hình ảnh.

Trong quá trình phát triển dự án "Paradox Caliber", tác giả đã tham khảo logic xây dựng hệ thống từ các nguồn tài liệu mở và mua bộ tài nguyên mã nguồn tương ứng để phục vụ nghiên cứu. Tuy nhiên, quá trình triển khai thực tế gặp phải hai rào cản kỹ thuật lớn:

- **Xung đột cấu trúc hệ thống:** Bộ kiến trúc mã nguồn gốc được xây dựng như một dự án độc lập, không phải là một mô-đun có tính đóng gói. Điều này đồng nghĩa với việc không thể tích hợp trực tiếp vào dự án hiện tại. Tác giả bắt buộc phải bóc tách mã nguồn, trích xuất các khối logic lõi và cấu hình lại toàn bộ để ghép nối an toàn vào Paradox Caliber.
- **Góc nhìn máy quay không tương thích:** Bộ tài nguyên gốc được thiết kế dành riêng cho game bắn súng góc nhìn thứ nhất, trong khi Paradox Caliber là một tựa game bắn súng góc nhìn thứ ba. Sự sai lệch hệ quy chiếu máy quay khiến các thuật toán nội suy đường đạn để khởi tạo cổng và thuật toán tính toán góc nhìn xuyên qua cổng ảo bị sai lệch hoàn toàn.

0.1.2 Phân tích và làm chủ logic hoạt động của BP_Portal

Để giải quyết bài toán trên, thay vì tái sử dụng rập khuôn, tác giả đã tiến hành kỹ thuật dịch ngược lớp trung tâm BP_Portal để nắm bắt trọn vẹn thuật toán. Các luồng xử lý chính được tác giả phân tách bao gồm:

- **Khối kiểm tra bề mặt hợp lệ (CheckSurface):** Hệ thống sinh ra các tia quét vô hình để tính toán va chạm lên bề mặt hình học của môi trường. Khối logic này trích xuất thông số pháp tuyến từ mặt phẳng, từ đó đánh giá xem không gian bao quanh có đủ độ rộng và độ phẳng để khởi tạo một cánh cổng mà không bị xuyên lấp hay không.

- **Khởi hiệu chỉnh tọa độ cổng (CanPlacePortal & OffsetDummyPortal):** Khi xác định được điểm hợp lệ, hệ thống sử dụng một cổng ảo làm mẫu chiếu. Lệnh Offset thực thi các phép tịnh tiến nhỏ nhằm dịch chuyển cánh cổng ra xa các ranh giới rào cản gần sát, đảm bảo hộp va chạm của cổng hoạt động an toàn.
- **Khởi cập nhật máy quay phản chiếu (UpdateCameras & UpdateCaptureM):** Để tạo ra ảo giác hai không gian liên thông, một thành phần giả lập máy quay được đặt ở cổng đích. Khối UpdateCameras liên tục mượn dữ liệu từ hệ thống quản lý để trích xuất vị trí, thực hiện tính toán khoảng cách vector tương đối, sau đó truyền phép biến đổi đó sang góc ống kính ở cổng đích.
- **Khởi thực thi bước nhảy không gian (JumpPortalM & GetTargetPoint):** Khi hộp va chạm của nhân vật tiến vào cổng, khối lệnh này tức thời lấy Vector từ hàm đích đến, triệt tiêu hộp va chạm cũ và dịch chuyển nhân vật sang tọa độ cổng đích; đồng thời xoay lại phương của Vector vận tốc giúp bảo toàn đà di chuyển vật lý của nhân vật.

0.1.3 Thiết kế thành phần vũ khí và Tối ưu hóa nhập liệu

Để đáp ứng kiến trúc mô-đun hóa, tiến trình xử lý tín hiệu gọi cổng được tách riêng vào lớp BP_Weapon_Component và tích hợp như một thành phần phụ trợ của nhân vật.

- **Ghi đè tín hiệu điều khiển:** Dựa vào cốt lõi của hệ thống nhập liệu nâng cao, cơ chế ngắt bắn cổng được đưa vào một ngữ cảnh điều khiển riêng biệt. Khi người chơi nhấn phím X, hệ thống sẽ gọi hàm thêm ngữ cảnh kèm theo độ ưu tiên cao hơn. Qua đó, thao tác chuột trái/phải thông thường sẽ tự động chuyển đổi thành hành động kích hoạt súng mở cổng. Cấu trúc này giúp cô lập thao tác triệt để mà không cần lạm dụng các biến điều kiện trạng thái phức tạp.
- **Thuật toán nội suy đường đạn (GetAmmoPoint & ShootGun):** Thuật toán lấy hướng xoay của ống kính làm quy chiếu gốc thay vì mốc nòng súng. Tiếp đó, hệ thống chiếu một tia thẳng về phía trước để truy xuất điểm chạm hợp lệ trên tường.
- **Mô phỏng đạn năng lượng (GetBullet & FlyBullet):** Thay vì mở cổng lập tức ngay khi kích hoạt, hệ thống sẽ khởi tạo một đối tượng đường đạn. Khối logic liên tục tính toán khoảng cách và cập nhật vị trí tạo hiệu ứng thị giác đạn bay tịnh tiến trong không gian tới vị trí đích.
- **Khởi tạo hộp va chạm cổng đích (GetPortalPlaceLocation):** Tại thời

khắc đường đạn và chạm, logic truyền dữ liệu điểm chạm về lớp gốc để khởi tạo cổng không gian, hoàn tất chu trình thuật toán.

Việc phân tích rõ ràng luồng thuật toán cốt lõi là tiền đề kỹ thuật vững chắc, đảm bảo quá trình tái cấu trúc không làm phá vỡ logic nền tảng, từ đó tạo nền móng cho phần đóng góp tùy biến quan trọng nhất: Chuyển đổi cơ chế hiển thị và bắn cổng sang góc nhìn thứ ba.

0.1.4 Tùy biến và giải quyết vấn đề góc nhìn thứ ba

Như đã đề cập, bộ mã nguồn nguyên bản được thiết kế cho game bắn súng góc nhìn thứ nhất, dẫn đến hàng loạt lỗi biên dịch và sai lệch logic khi áp dụng vào nhân vật góc nhìn thứ ba. Quá trình chuyển đổi và chuẩn hóa mã nguồn được tác giả thực hiện qua các bước then chốt sau:

1. Loại bỏ phụ thuộc cục bộ của góc nhìn thứ nhất:

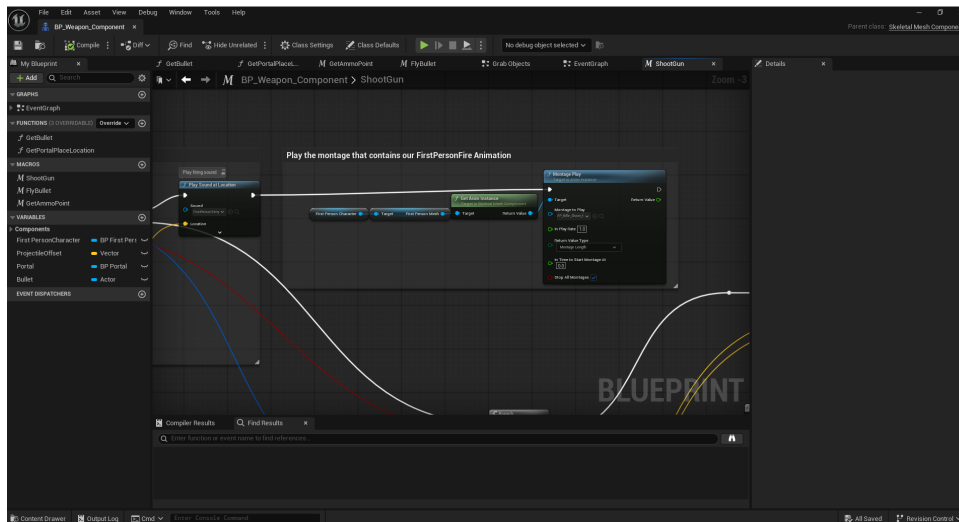
Trong kiến trúc gốc, logic hệ thống bị gắn chặt với các thành phần lưới mô hình và các hàm chạy hoạt ảnh dành riêng cho góc nhìn thứ nhất. Tác giả đã tiến hành cắt bỏ toàn bộ các nút lệnh hoạt ảnh này, tách rời BP_Weapon_Component thành một hệ thống chức năng độc lập gắn ẩn vào nhân vật, thay vì đóng vai trò là một mô hình súng vật lý. Điều này không chỉ giải quyết triệt để lỗi biên dịch mà còn tối ưu hóa tài nguyên hệ thống (xem Hình 0.1).

2. Hiệu chỉnh thuật toán dò tia cho góc nhìn thứ ba:

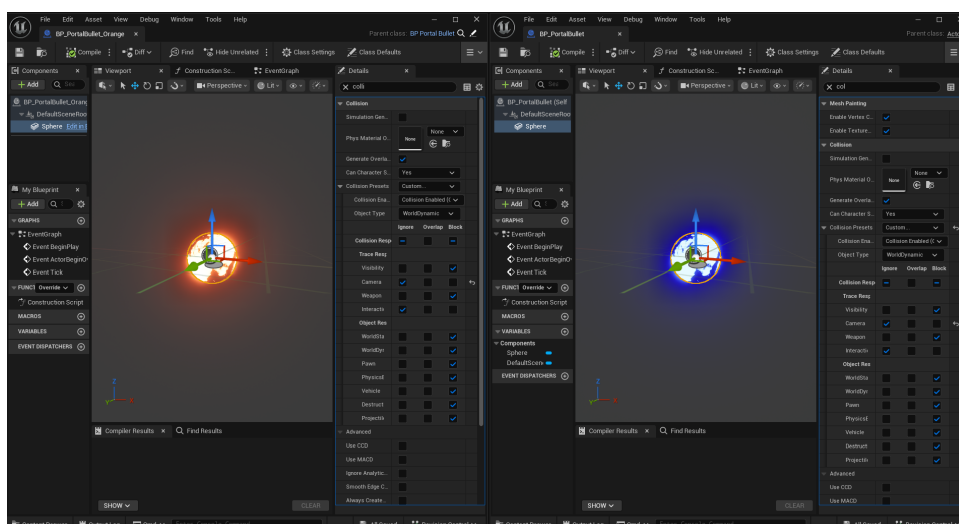
Một sai lầm phổ biến khi chuyển góc nhìn là thay đổi mốc bắn tia dò tìm từ máy quay sang tọa độ của nhân vật. Do máy quay góc nhìn thứ ba đặt lệch qua vai, tâm ngắm màn hình và mũi súng không nằm trên cùng một đường thẳng. Để đảm bảo độ chính xác, tác giả quyết định giữ nguyên thuật toán sử dụng tọa độ và góc xoay của máy quay. Tia chiếu vô hình sẽ xuất phát từ máy quay, xuyên qua giữa màn hình để tìm điểm chạm trên tường, sau đó định hướng quỹ đạo đạn bay từ nòng súng tới điểm chạm đó.

3. Khắc phục lỗi nội suy máy quay:

Trong quá trình thử nghiệm làm chậm thời gian, màn hình phát sinh hiện tượng giật đột ngột mỗi khi kích hoạt vũ khí. Nguyên nhân được xác định do xung đột vật lý của thanh kết nối máy quay. Khi viên đạn sinh ra tại nòng súng, hộp va chạm của nó mặc định chặn máy quay, khiến thanh kết nối tự động co ngắn lại. Giải pháp kỹ thuật được áp dụng là chuyển đổi thiết lập va chạm của viên đạn sang tùy chỉnh và thiết lập trạng thái bỏ qua đối với máy quay và nhân vật (xem Hình 0.2).



Hình 0.1: Cấu trúc sơ đồ logic sau khi loại bỏ các phụ thuộc hoạt ảnh góc nhìn thứ nhất.

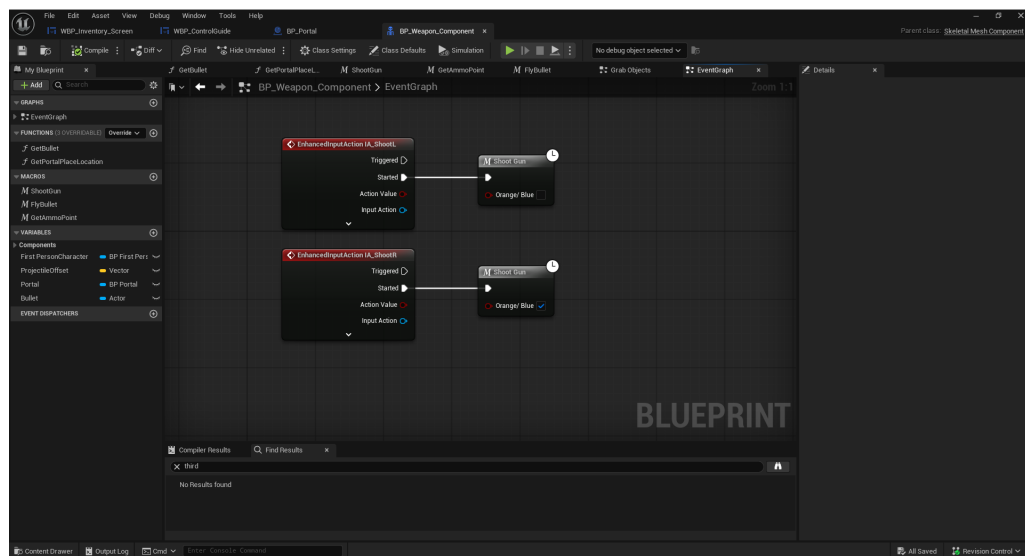


Hình 0.2: Cấu hình tùy chỉnh vật lý của đối tượng đạn nhằm khắc phục xung đột với thanh kết nối máy quay.

0.1.5 Tối ưu hóa độ ổn định của hệ thống nhập liệu

Quá trình áp dụng hệ thống nhập liệu nâng cao phát sinh một số vấn đề về độ ổn định tín hiệu, được tác giả xử lý như sau:

- **Xử lý lỗi kích hoạt lặp tín hiệu:** Khi nhấn phím X để chuyển trạng thái súng, chân thực thi mặc định liên tục gửi tín hiệu theo từng khung hình dựa trên thời gian giữ phím, khiến hệ thống chuyển đổi thiếu ổn định. Tác giả đã hiệu chỉnh lại luồng dữ liệu sang chân thực thi khởi đầu, đảm bảo lệnh chỉ được thực thi một lần duy nhất cho mỗi xung lực nhấn.
- **Chặn tín hiệu nền:** Để giải quyết vấn đề tín hiệu ngắt bắn của súng mở cổng bị kích hoạt chung với hiệu ứng ngắt của vũ khí cơ bản, tác giả kích hoạt tính năng tiêu thụ tín hiệu. Kết hợp với thiết lập độ ưu tiên cao, các tín hiệu chuột trái/phải được xử lý và tiêu hủy hoàn toàn trong luồng súng mở cổng, triệt tiêu sự can thiệp ngoài ý muốn.



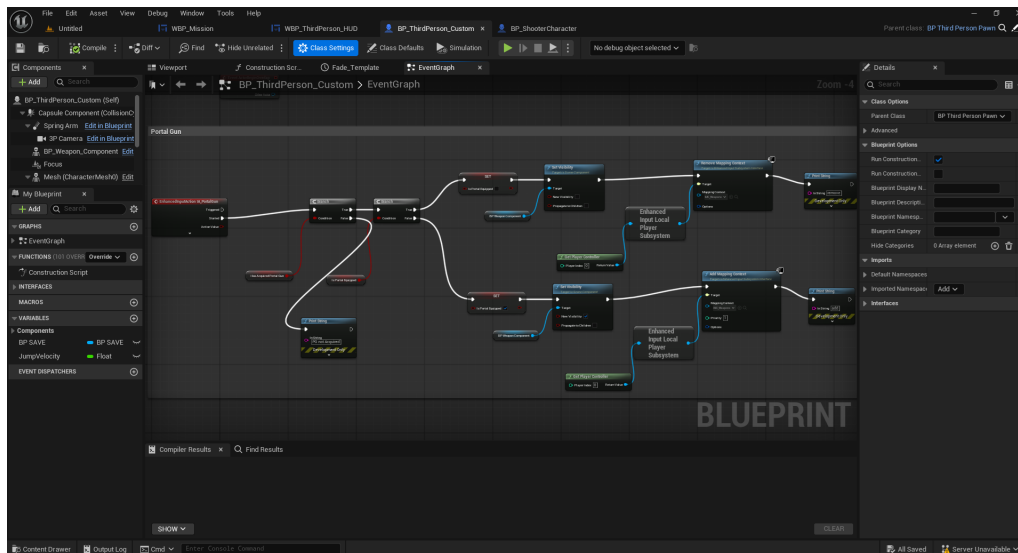
Hình 0.3: Sơ đồ luồng sự kiện của thành phần vũ khí được kết nối qua chân thực thi khởi đầu để ngăn chặn lỗi đa luồng.

0.1.6 Khắc phục lỗi tương tác và giao diện giao tiếp dữ liệu

Sau khi hoàn thiện cơ chế bắn, tác giả tiếp tục giải quyết các rào cản liên quan đến tương tác vật lý bề mặt và quá trình sinh công.

1. Quản lý bề mặt vật lý:

Nút logic kiểm tra bề mặt của tài nguyên gốc liên tục báo lỗi do thiếu hụt biến môi trường. Tác giả đã tái thiết lập thông số trong phần cài đặt dự án. Để gán thuộc tính này cho các cấu trúc tường phức tạp mà không ảnh hưởng hiệu năng, phương pháp ghi đè vật liệu được áp dụng. Đồng thời, thuật toán tia quét được thiết lập tùy chọn độ chính xác cao nhằm nội suy chính xác vật



Hình 0.4: Sơ đồ logic xử lý tín hiệu đầu vào nhằm bật/tắt ghi đè ngữ cảnh điều khiển.

liệu từ bề mặt lưới đa giác thay vì hộp va chạm thô.

2. Xử lý lỗi rỗng khởi tạo đối tượng:

Quá trình gỡ lỗi cho thấy biến chứa mục tiêu cổng luôn ở trạng thái rỗng. Nguyên nhân xuất phát từ kỹ thuật tái sử dụng đối tượng: cổng không được khởi tạo mới mà chỉ dịch chuyển một thực thể có sẵn. Giải pháp được đưa ra là bổ sung hàm gọi quét đối tượng tại sự kiện bắt đầu trò chơi nhằm thiết lập liên kết với các đối tượng tĩnh trên bản đồ (xem Hình 0.5).

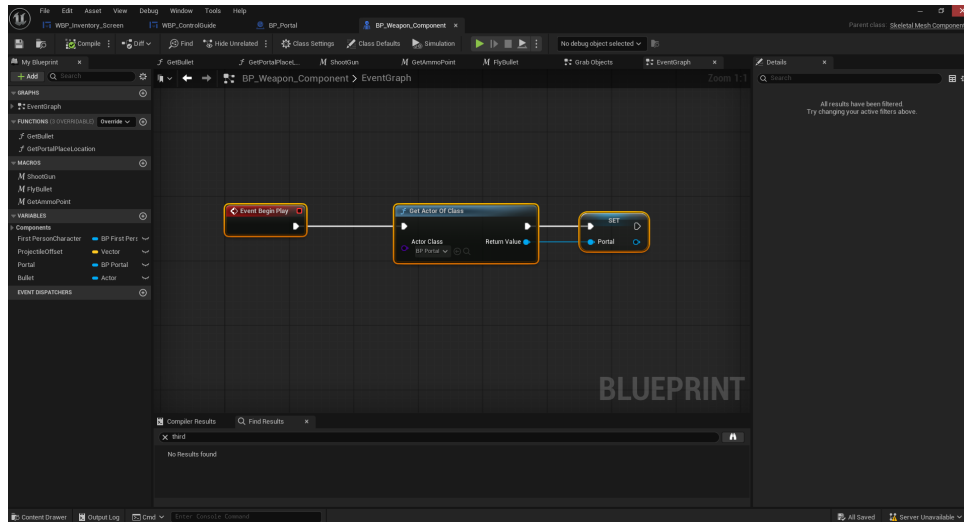
3. Tích hợp giao diện giao tiếp dữ liệu:

Vấn đề cuối cùng được khắc phục là hiện tượng nhân vật đi xuyên qua cổng mà không được dịch chuyển. Qua phân tích luồng dịch chuyển, hệ thống yêu cầu đối tượng va chạm phải triển khai một giao diện giao tiếp dữ liệu bắt buộc. Sau khi tích hợp giao diện này vào phần cài đặt của nhân vật và chuyển đổi khối logic tính toán góc xoay, hệ thống dịch chuyển không gian đã hoạt động chính xác (xem Hình 0.6).

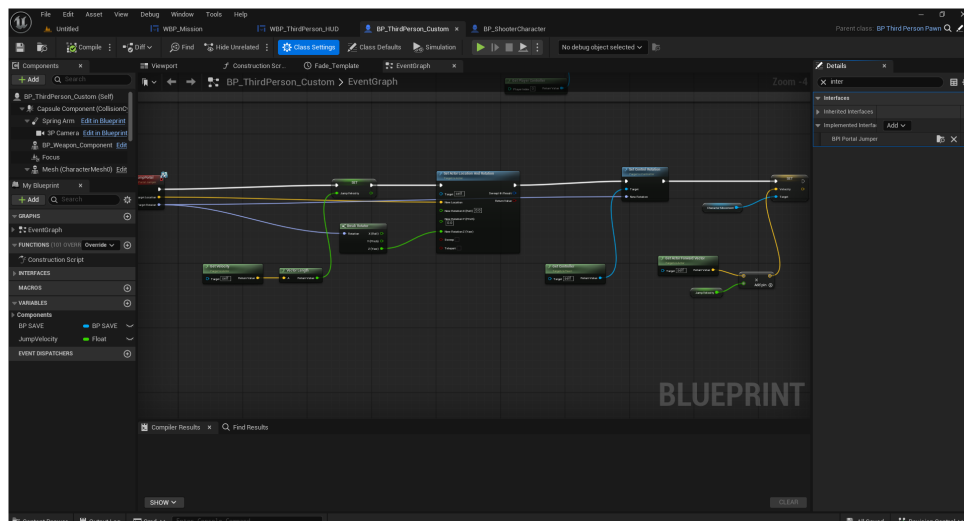
0.2 Kiến trúc hệ thống nhiệm vụ mở rộng linh hoạt

0.2.1 Dẫn dắt vấn đề và Tư duy thiết kế

Trong giai đoạn đầu phát triển, hệ thống kiểm soát nhiệm vụ được xây dựng theo hướng nguyên khối. Toàn bộ logic đếm số lượng mục tiêu, kiểm tra va chạm, và cập nhật giao diện người dùng bị đóng gói cứng nhắc vào một lớp duy nhất và ép kiểu trực tiếp tới nhân vật. Cấu trúc này bộc lộ nhược điểm nghiêm trọng: không thể tái sử dụng, dễ gây xung đột biến trạng thái và khó mở rộng cho các kịch bản màn chơi phức tạp.



Hình 0.5: Phương pháp gắn liên kết đối tượng thông qua sự kiện khởi đầu trò chơi nhằm khắc phục lỗi rỗng.



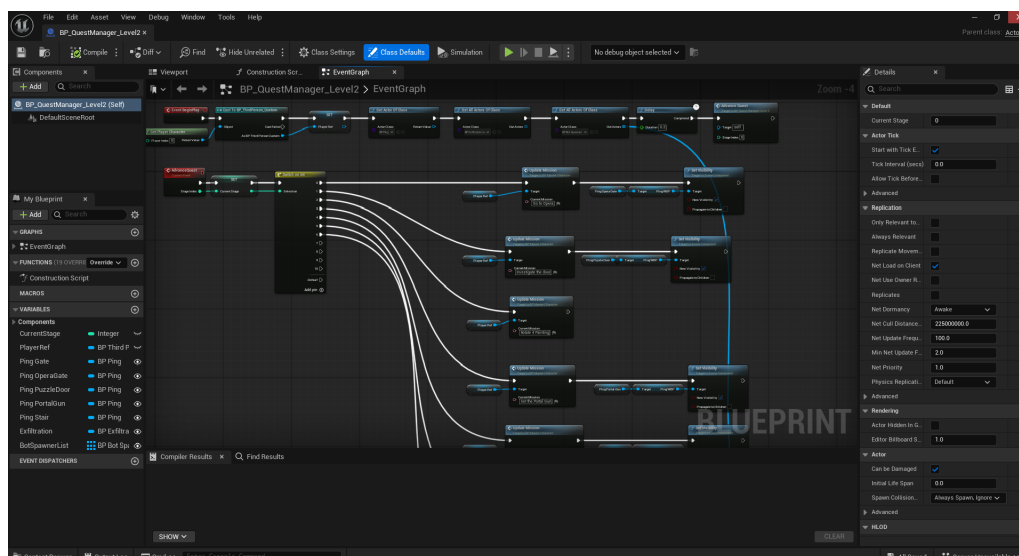
Hình 0.6: Tích hợp giao diện giao tiếp dữ liệu vào lớp nhân vật nhằm đáp ứng điều kiện nhận diện dịch chuyển của hệ thống.

Để khắc phục, tác giả đã tái cấu trúc hệ thống sang mô hình quản lý theo trạng thái, tách biệt khối điều phối trung tâm và điểm kích hoạt. Giải pháp này cung cấp khả năng hoạt động độc lập, tính mở rộng cao và cho phép thiết kế màn chơi linh hoạt thông qua dữ liệu tham số.

0.2.2 Xây dựng khối điều phối trung tâm

Một đối tượng ẩn mang tên BP_QuestManager_Level2 được khởi tạo để đóng vai trò trung tâm kiểm soát luồng sự kiện của toàn bộ màn chơi.

- **Cơ chế rẽ nhánh trạng thái:** Tác giả sử dụng biến số nguyên để định vị tiến trình. Một sự kiện tùy chỉnh được tạo ra để tiếp nhận tham số đầu vào là chỉ số nguyên. Nút rẽ nhánh đảm nhiệm việc chuyển luồng thực thi tương ứng với từng giai đoạn kịch bản (Ví dụ: 0 - Vào khu vực, 1 - Mở cửa, 2 - Giải đố).
- **Quản lý danh sách tham chiếu:** Các điểm chỉ dẫn và vùng thoát ly được khai báo thành các biến tham chiếu. Tại mỗi nhánh trạng thái, hệ thống quản lý sẽ thực thi lệnh cập nhật văn bản giao diện và điều khiển hiển thị các điểm tương ứng.



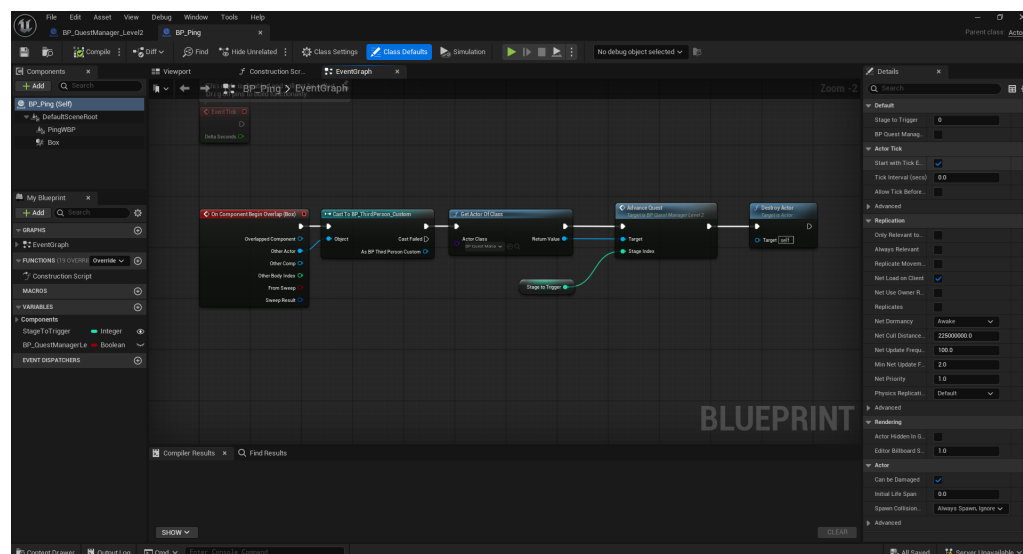
Hình 0.7: Cơ chế rẽ nhánh trạng thái nhiệm vụ dựa trên biến số nguyên bên trong hệ thống quản lý.

0.2.3 Tối ưu hóa điểm chỉ dẫn và vùng kích hoạt

Điểm chỉ dẫn nhiệm vụ được nâng cấp thành một đối tượng đa chức năng, kết hợp hiển thị giao diện trực quan và vùng kích hoạt sự kiện.

- **Giao diện không gian màn hình:** Để biểu tượng chỉ dẫn luôn hướng về máy quay và hiển thị xuyên vật thể, tác giả sử dụng thành phần giao diện với chế độ dựng hình trong không gian màn hình.
- **Cơ chế kích hoạt và giải phóng bộ nhớ:** Đối tượng chứa một hộp va chạm có

thể tùy chỉnh kích thước. Khi phát sinh va chạm, đối tượng truy xuất biến mục tiêu, truyền tín hiệu tới hệ thống quản lý nhiệm vụ, sau đó gọi lệnh tự hủy để giải phóng vùng nhớ và ngăn chặn lỗi kích hoạt lặp.



Hình 0.8: Cấu trúc thành phần của điểm chỉ dẫn và luồng sự kiện cập nhật tiến trình nhiệm vụ khi phát sinh va chạm.

0.2.4 Xử lý biệt lệ giao diện và Tích hợp chuỗi sự kiện

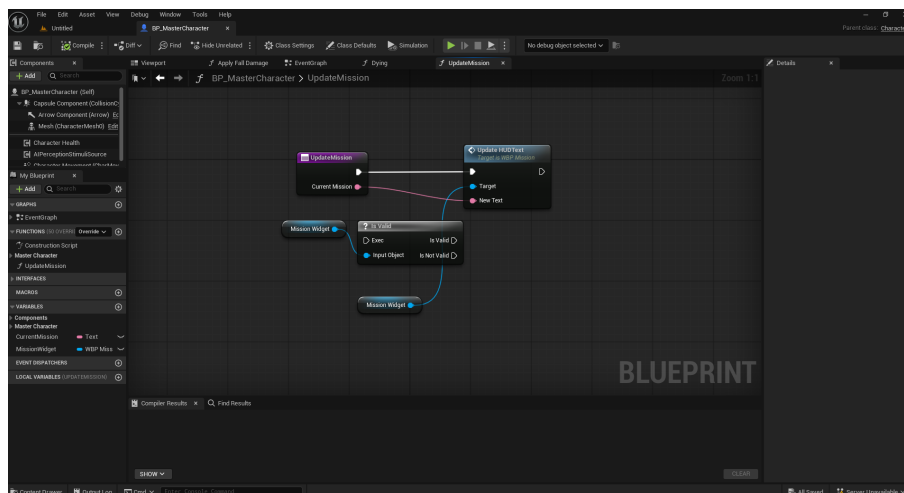
Trong thực tế vận hành, việc tách rời hệ thống sinh ra lỗi thực thi khi cập nhật giao diện người dùng. Nguyên nhân bắt nguồn từ việc hệ thống quản lý gửi lệnh cập nhật giao diện ngay khoảnh khắc màn chơi vừa khởi tạo, khi lớp giao diện của nhân vật chưa kịp sinh ra. Tác giả khắc phục bằng cơ chế phòng lỗi: sử dụng nút kiểm tra tính hợp lệ trước khi cập nhật văn bản; nếu giao diện chưa tồn tại, hệ thống sẽ thực thi lệnh khởi tạo để bù đắp độ trễ đồng bộ.

Ngoài các điểm chạm thông thường, hệ thống cho phép tích hợp tiến trình từ các sự kiện ngoại vi. Cụ thể, các đối tượng tương tác như cánh cửa giải đố hay vật phẩm có thể chủ động giao tiếp với hệ thống quản lý để báo cáo hoàn thành mục tiêu.

0.3 Thiết kế hệ thống khởi tạo kẻ địch

0.3.1 Dẫn dắt vấn đề

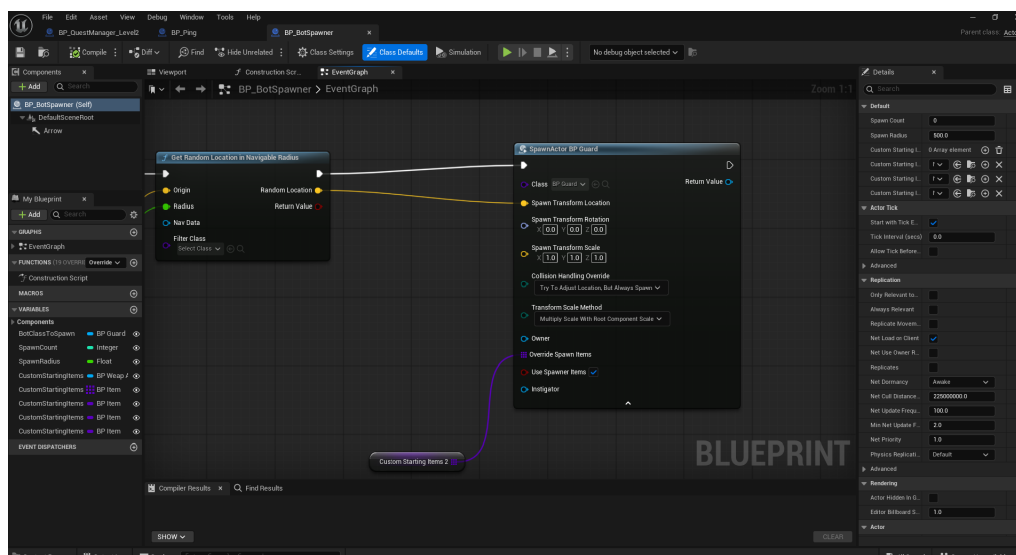
Để gia tăng thử thách ở giai đoạn cuối màn chơi, kịch bản yêu cầu người chơi phải đối mặt với một toán lính canh. Thay vì đặt trước các nhân vật do máy điều khiển này trên bản đồ, tác giả đã phát triển một hệ thống khởi tạo kẻ địch theo kịch bản linh hoạt, cho phép cấu hình tọa độ, số lượng và trang bị trực tiếp trên giao diện thiết kế.



Hình 0.9: Cơ chế phòng lỗi sử dụng nút kiểm tra tính hợp lệ trước khi cập nhật giao diện người dùng để tránh lỗi đồng bộ.

0.3.2 Thuật toán phân bố vị trí an toàn trên lưới điều hướng

Một lỗi phổ biến khi sử dụng vòng lặp để khởi tạo nhiều đối tượng tại cùng một tọa độ là hiện tượng các mô hình bị chồng chéo hoặc văng khỏi bản đồ do lực đẩy vật lý. Để giải quyết triệt để, hệ thống tích hợp thuật toán tìm kiếm tọa độ không gian ngẫu nhiên. Thông qua việc kết hợp với lưới điều hướng, hệ thống sẽ nội suy các điểm ngẫu nhiên trong bán kính cho trước, đảm bảo thực thể sinh ra luôn nằm trên bề mặt di chuyển hợp lệ.

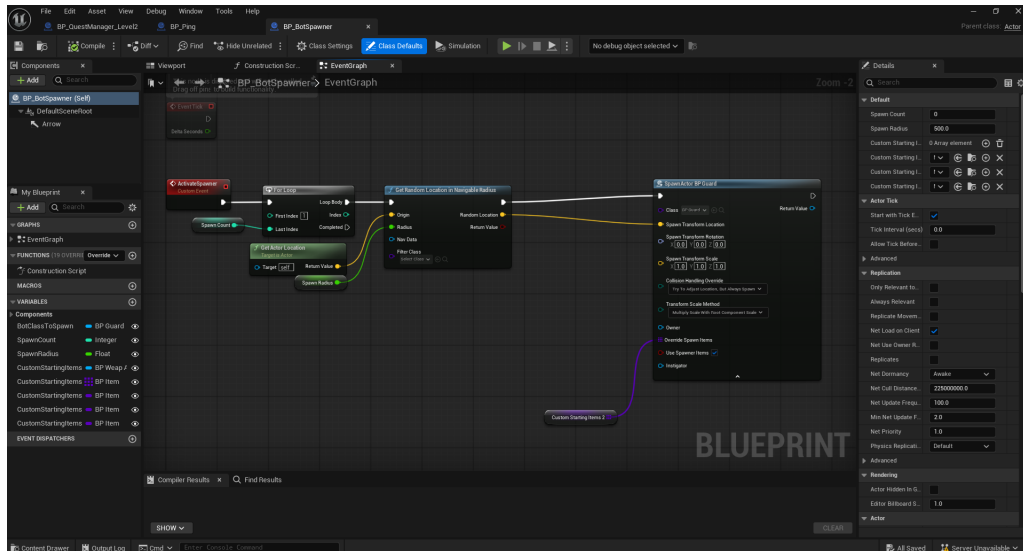


Hình 0.10: Thuật toán phân bố vị trí ngẫu nhiên trên lưới điều hướng trong bán kính cho trước nhằm tránh lỗi chồng chéo vật lý.

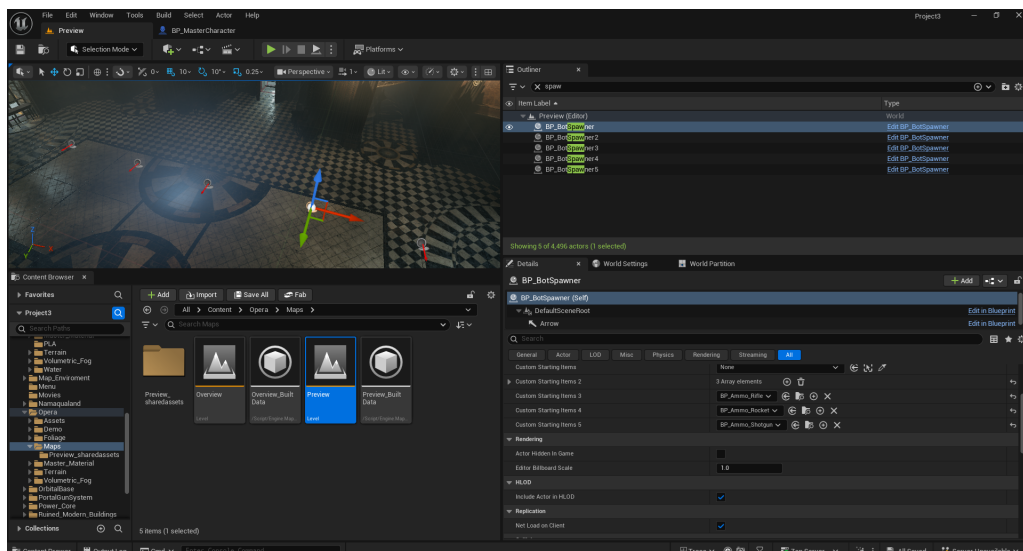
0.3.3 Quản lý trạng bị động qua thành phần chức năng

Một cải tiến kỹ thuật đáng chú ý là cơ chế truyền dữ liệu trạng bị từ hệ thống khởi tạo vào túi đồ của nhân vật do máy điều khiển. Thay vì khởi tạo với trạng bị

mặc định cứng nhắc, hệ thống khởi tạo tích hợp một biến mảng chứa danh sách các vật phẩm tùy chỉnh. Sau khi gọi lệnh tạo thực thể thành công, hệ thống ép kiểu để truy cập thành phần chứa túi đồ bên trong nhân vật. Thông qua nút lệnh thiết lập, mảng dữ liệu trang bị được truyền trực tiếp vào thành phần này. Cơ chế này cho phép thiết lập nhiều hệ thống khởi tạo với các cấu hình lính canh khác nhau (như lính súng lục, lính súng trường) mà không cần nhân bản lớp nhân vật gốc.



Hình 0.11: Luồng logic truyền dữ liệu trang bị động từ hệ thống khởi tạo vào thành phần túi đồ của kẻ địch.



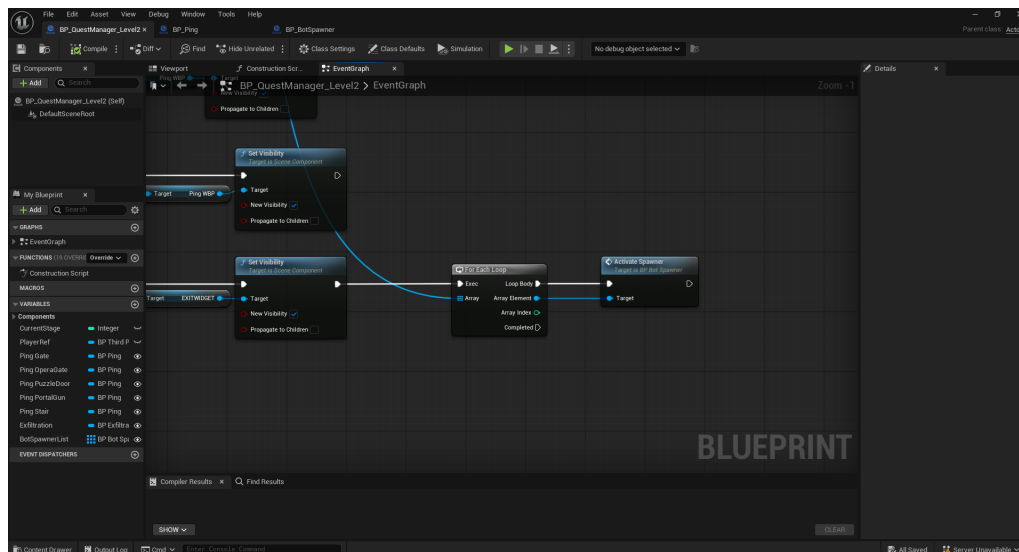
Hình 0.12: Cấu hình tham số mảng trang bị tùy chỉnh trên giao diện thiết kế cho từng hệ thống khởi tạo.

0.3.4 Khắc phục lỗi điều khiển trí tuệ nhân tạo và Liên kết kịch bản

Trong quá trình thử nghiệm, phát sinh hiện tượng nhân vật do máy điều khiển được khởi tạo bằng mã lệnh rơi vào trạng thái bất động, khác với các nhân vật được đặt thủ công. Phân tích vòng đời đối tượng cho thấy, lớp đối tượng điều khiển mặc

định không tự động liên kết với hệ thống trí tuệ nhân tạo khi được sinh ra qua mã lệnh. Tác giả đã can thiệp vào cài đặt của lớp lính gác, chuyển đổi thông số tự động chiếm quyền điều khiển sang trạng thái bao gồm cả đặt trong thế giới hoặc sinh ra qua lệnh, giúp hệ thống lập tức kiểm soát ngay khi xuất hiện.

Cuối cùng, để đồng bộ với hệ thống nhiệm vụ, hệ thống quản lý được khai báo một mảng chứa tham chiếu các điểm khởi tạo. Tại thời điểm xác định của nhiệm vụ, hệ thống sử dụng vòng lặp để truyền lệnh đánh thức toàn bộ các điểm này, tạo ra chuỗi sự kiện phục kích đúng với kịch bản thiết kế.



Hình 0.13: Vòng lặp duyệt mảng trong hệ thống quản lý nhiệm vụ để kích hoạt đồng loạt chuỗi sự kiện xuất hiện kẻ địch.